

Directed Cycle Detection

Directed Graphs, Cycle Detection (DFS Colours) · Mentor-Led Lab
(Student Edition)

Developed by TalenciaGlobal

Difficulty ★★★ Advanced

Problem

Detect whether a directed graph contains a cycle, and count its sources (in-degree 0). You are given starter code with two functions. `has_cycle` is almost complete but has a bug, and `count_sources` is an empty stub. Fix the bug and implement the stub.

Directed cycle detection needs a three-colour DFS: a cycle exists only when the search reaches a vertex still on the current recursion stack.

What to Compute

- **Has Cycle:** Yes if the directed graph contains a cycle, otherwise No.
- **Sources:** the number of vertices with in-degree 0.

Input / Output Format

Input: Line 1: N M . Next M lines: a directed edge u v .

Output: Two lines: Has Cycle and Sources.

Examples

Example 1

Input: 4 4 0 1 0 2 1 3 2 3

Output:

Has Cycle: No
Sources: 1

Explanation: This diamond DAG (0 to 1, 0 to 2, both to 3) has no cycle; only vertex 0 has in-degree 0.

Example 2

Input: 3 3 0 1 1 2 2 0

Output:

Has Cycle: Yes
Sources: 0

Explanation: The edges 0 to 1 to 2 to 0 form a cycle, and every vertex has an incoming edge, so there are no sources.

Constraints

- A cycle exists when DFS reaches a vertex still on the recursion stack (grey).
- A finished (black) vertex is not a cycle.
- A source has in-degree 0.

Your Task

- Task 1 - Fix the bug. `has_cycle` is flagged with a comment, but the bug's location is not given. Find the single bug so a cycle is reported only for a grey (on-stack) vertex, not for any already-visited one.
- Task 2 - Implement `count_sources` from scratch: the number of vertices with in-degree 0.

Starter Code

Begin from the code below. One function carries a single bug (flagged by a comment, with no hint to its location); the other is an empty stub you must implement.

Python

```
import sys
import heapq
from collections import deque

def read_ints():
    return [int(x) for x in sys.stdin.read().split()]

def build_directed(data):
    n = data[0]
    m = data[1]
    adj = [set() for _ in range(n)]
    indeg = [0] * n
    idx = 2
    for _ in range(m):
        u = data[idx]
        v = data[idx + 1]
        idx += 2
        if v not in adj[u]:
            adj[u].add(v)
            indeg[v] += 1
    adj = [sorted(s) for s in adj]
    return n, m, adj, indeg, idx

# BUG: this function contains a single bug. A directed cycle exists only when a DFS
# meets a vertex still on the current recursion stack - not merely one already finished.
def has_cycle(n, adj):
```

```

visited = [False] * n
def dfs(u):
    visited[u] = True
    for w in adj[u]:
        if visited[w]:
            return True
        if dfs(w):
            return True
    return False
for s in range(n):
    if not visited[s]:
        if dfs(s):
            return True
return False

# TODO: implement this. It must count the source vertices (in-degree 0).
# Currently a stub.
def count_sources(n, indeg):
    return 0

def main():
    data = read_ints()
    n, m, adj, indeg, idx = build_directed(data)
    print("Has Cycle: " + ("Yes" if has_cycle(n, adj) else "No"))
    print("Sources: " + str(count_sources(n, indeg)))

main()

```

C++

```

// Lab 4 - Directed Cycle Detection (STARTER)
#include <iostream>
#include <vector>
#include <string>
#include <set>
#include <queue>
#include <algorithm>
using namespace std;

vector<long long> readInts() {
    vector<long long> data;
    long long x;
    while (cin >> x) {
        data.push_back(x);
    }
    return data;
}

vector<vector<int>>> buildDirected(const vector<long long>& data, int& n, int& m,
vector<int>& indeg, int& idx) {
    n = (int) data[0];
    m = (int) data[1];
    vector<set<int>>> tmp(n);
    indeg.assign(n, 0);
    idx = 2;
    for (int i = 0; i < m; i++) {
        int u = (int) data[idx];

```

```

        int v = (int) data[idx + 1];
        idx += 2;
        if (tmp[u].find(v) == tmp[u].end()) {
            tmp[u].insert(v);
            indeg[v]++;
        }
    }
    vector<vector<int>> adj(n);
    for (int i = 0; i < n; i++) {
        adj[i] = vector<int>(tmp[i].begin(), tmp[i].end());
    }
    return adj;
}

vector<bool> visited;

// BUG: this function contains a single bug. A directed cycle exists only when a DFS
// meets a vertex still on the current recursion stack - not merely one already finished.
bool dfs(int u, const vector<vector<int>>& adj) {
    visited[u] = true;
    for (int w : adj[u]) {
        if (visited[w]) {
            return true;
        }
        if (dfs(w, adj)) {
            return true;
        }
    }
    return false;
}

bool hasCycle(int n, const vector<vector<int>>& adj) {
    visited.assign(n, false);
    for (int s = 0; s < n; s++) {
        if (!visited[s]) {
            if (dfs(s, adj)) {
                return true;
            }
        }
    }
    return false;
}

// TODO: implement this. It must count the source vertices (in-degree 0).
// Currently a stub.
int countSources(int n, const vector<int>& indeg) {
    return 0;
}

int main() {
    vector<long long> data = readInts();
    int n, m, idx;
    vector<int> indeg;
    vector<vector<int>> adj = buildDirected(data, n, m, indeg, idx);
    cout << "Has Cycle: " << (hasCycle(n, adj) ? "Yes" : "No") << "\n";
    cout << "Sources: " << countSources(n, indeg) << "\n";
    return 0;
}

```

```
}
```

Java

```
// Lab 4 - Directed Cycle Detection (STARTER)
```

```
import java.util.*;
```

```
public class Main {
```

```
static int N, M;
```

```
static List<Long> DATA;
```

```
static int EDGE_END;
```

```
static int[] INDEG;
```

```
static List<List<Integer>> buildDirected(Scanner sc) {
```

```
    List<Long> data = new ArrayList<>();
```

```
    while (sc.hasNextLong()) {
```

```
        data.add(sc.nextLong());
```

```
    }
```

```
    DATA = data;
```

```
    N = (int) (long) data.get(0);
```

```
    M = (int) (long) data.get(1);
```

```
    List<TreeSet<Integer>> tmp = new ArrayList<>();
```

```
    for (int i = 0; i < N; i++) {
```

```
        tmp.add(new TreeSet<>());
```

```
    }
```

```
    INDEG = new int[N];
```

```
    int idx = 2;
```

```
    for (int i = 0; i < M; i++) {
```

```
        int u = (int) (long) data.get(idx);
```

```
        int v = (int) (long) data.get(idx + 1);
```

```
        idx += 2;
```

```
        if (!tmp.get(u).contains(v)) {
```

```
            tmp.get(u).add(v);
```

```
            INDEG[v]++;
```

```
        }
```

```
    }
```

```
    EDGE_END = idx;
```

```
    List<List<Integer>> adj = new ArrayList<>();
```

```
    for (int i = 0; i < N; i++) {
```

```
        adj.add(new ArrayList<>(tmp.get(i)));
```

```
    }
```

```
    return adj;
```

```
}
```

```
static boolean[] visited;
```

```
// BUG: this function contains a single bug. A directed cycle exists only when a DFS
```

```
// meets a vertex still on the current recursion stack - not merely one already finished.
```

```
static boolean dfs(int u, List<List<Integer>> adj) {
```

```
    visited[u] = true;
```

```
    for (int w : adj.get(u)) {
```

```
        if (visited[w]) {
```

```
            return true;
```

```
        }
```

```
        if (dfs(w, adj)) {
```

```
            return true;
```

```
        }
```

```
    }
    return false;
}

static boolean hasCycle(int n, List<List<Integer>> adj) {
    visited = new boolean[n];
    for (int s = 0; s < n; s++) {
        if (!visited[s]) {
            if (dfs(s, adj)) {
                return true;
            }
        }
    }
    return false;
}

// TODO: implement this. It must count the source vertices (in-degree 0).
// Currently a stub.
static int countSources(int n, int[] indeg) {
    return 0;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    List<List<Integer>> adj = buildDirected(sc);
    System.out.println("Has Cycle: " + (hasCycle(N, adj) ? "Yes" : "No"));
    System.out.println("Sources: " + countSources(N, INDEG));
}
}
```

Sample Run

Sample Input

```
4 4
0 1
0 2
1 3
2 3
```

Sample Output

```
Has Cycle: No
Sources: 1
```